JOBSPHERE

Job Portal & Recruitment System

Professional Project Documentation

Platform: Candidate + Recruitment Partner workspaces

Technology: React • Vite • Spring Boot • Java 17 • PostgreSQL

A functional recruitment platform for discovering opportunities, submitting applications, publishing job listings, and screening applicants.

Repository: github.com/bru-reddy/Job_Portal_And_Recruitment_System

Live frontend: jobsphere-frontend-2x4k.onrender.com

Contents

1. Executive Overview

2. Problem Statement & Objectives

3. Technology Stack

4. System Architecture

5. User Roles & Access Model

6. Candidate Workflow

7. Recruitment Partner Workflow

8. Company & Job Discovery

9. Build Your Skills

10. REST API & Data Model

11. Security, Validation & Demo Boundaries

12. Deployment Architecture

13. Interface Documentation

14. Testing & Validation

15. Future Enhancements

16. Conclusion & Project Links

1. Executive Overview

JobSphere is a full-stack job portal and recruitment system designed around two distinct workspaces: Candidate and Recruitment Partner. Candidates can discover opportunities, review eligibility and skills, submit applications, and track application progress. Recruitment Partners can publish job opportunities, manage listings, review applicants, and use skill relevance information during screening.

The implementation combines a responsive React/Vite frontend with a Spring Boot REST API and PostgreSQL persistence. The interface uses a consistent blue–violet visual language, card-based information architecture, clear calls to action, and responsive layouts for desktop and mobile usage.

The screenshots in this document are taken from the supplied JobSphere interface and representative frames extracted from the supplied screen recording. Technical descriptions are aligned with the implemented repository structure.

2. Problem Statement & Objectives

Traditional job-search workflows often split discovery, application submission, status tracking, and recruitment review across separate systems. JobSphere brings the core workflow into one application while preserving a clear boundary between candidate-facing actions and recruitment-facing actions.

Objective	Implementation focus
Opportunity discovery	Search by role, company, location, or skill and inspect job details.
Application workflow	Collect candidate details, education, experience, skills and additional information.
Application tracking	Expose application status and explanatory messages through the candidate dashboard.
Recruiter workflow	Create, review and monitor job postings and applicants.
Screening support	Calculate relevance from overlap between required job skills and candidate skills.
Learning support	Provide Learn and Practice paths with curated external resources.

3. Technology Stack

Layer	Technology	Role
Frontend	React 18, Vite 5, JavaScript, CSS	Responsive single-page user interface and client-side workflow states.
UI icons	Lucide React	Consistent interface iconography.
Backend	Spring Boot 3.3.5, Java 17	REST API, business logic, validation and persistence orchestration.
Persistence	Spring Data JPA / Hibernate	Object-relational mapping and repository access.
Database	PostgreSQL	Persistent users, jobs and applications.
Security	Spring Security, BCrypt	Authentication and password hashing.
Deployment	Render, Docker	Hosted frontend/backend deployment and containerized API workflow.

Repository structure follows a conventional separation of frontend, backend, controllers, services, repositories, models and DTOs. This supports maintainability and keeps UI concerns separate from API and persistence logic.

4. System Architecture

Candidate / Recruiter	React + Vite Frontend	Spring Boot REST API	PostgreSQL
Search jobs, apply, track status, create/manage jobs, review applicants.	Responsive screens, forms, navigation, API wrapper and local session state.	Controllers → services → repositories; authentication, validation and business rules.	Users, jobs, applications and related workflow data.

Primary workflow: User interface → REST endpoint → controller/service → JPA repository → PostgreSQL → API response → interface state update.

External job and learning sources remain outside the core database transaction. Job listings can retain source name and source URL, while learning resources open external curated destinations.

5. User Roles & Access Model

Role	Primary capabilities
Candidate	Discover jobs, inspect job requirements, submit applications, view application tracker, access learning resources.
Recruitment Partner	Create and manage job listings, view applicants, review candidate information and screening relevance.
Company / Admin	Backend role values exist in the data model, but the supplied frontend workflow exposes Candidate and Recruitment Partner workspaces.

6. Candidate Workflow

The candidate journey is structured as a continuous flow: authenticate → find an opportunity → inspect eligibility and skills → complete the application → submit → monitor the application tracker.

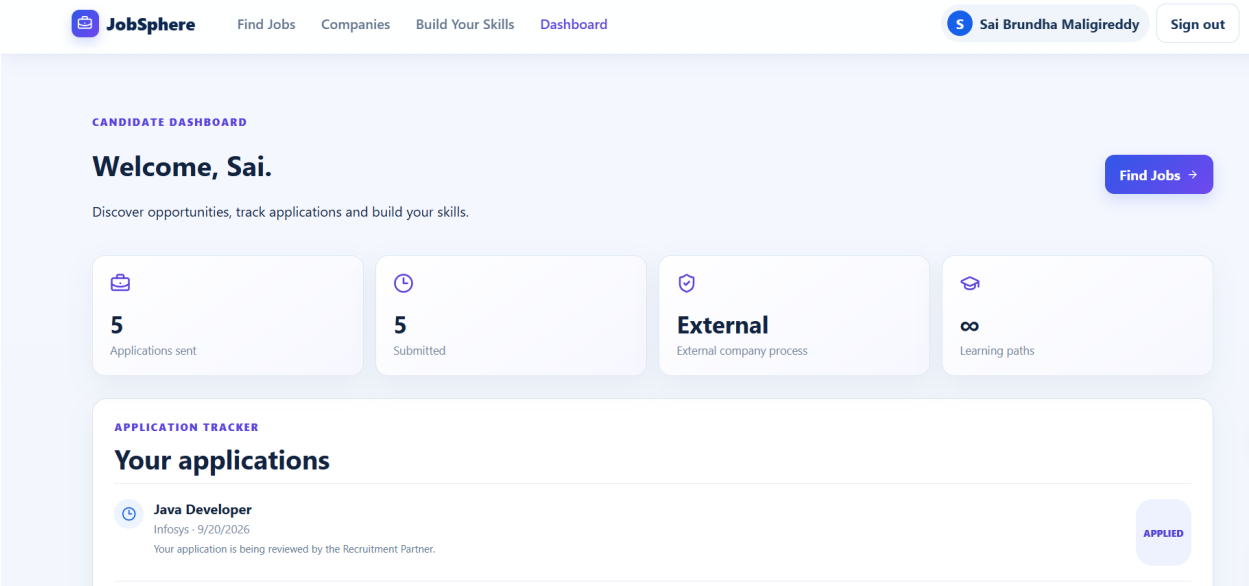


Figure 6.1 — Candidate Dashboard.

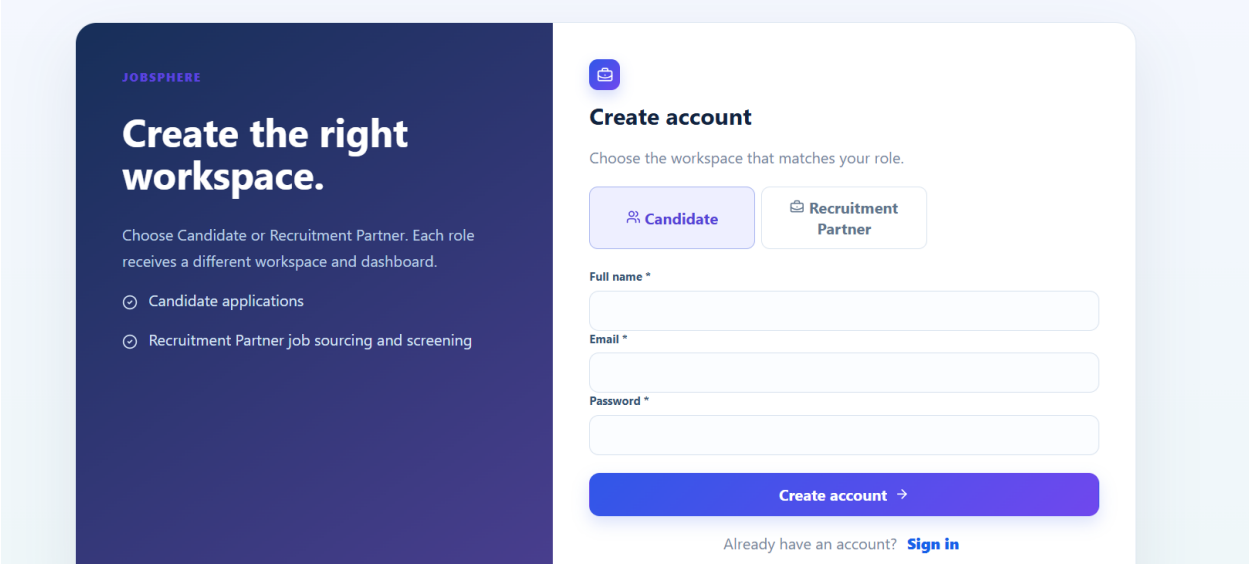


Figure 6.3 — Candidate Registration.

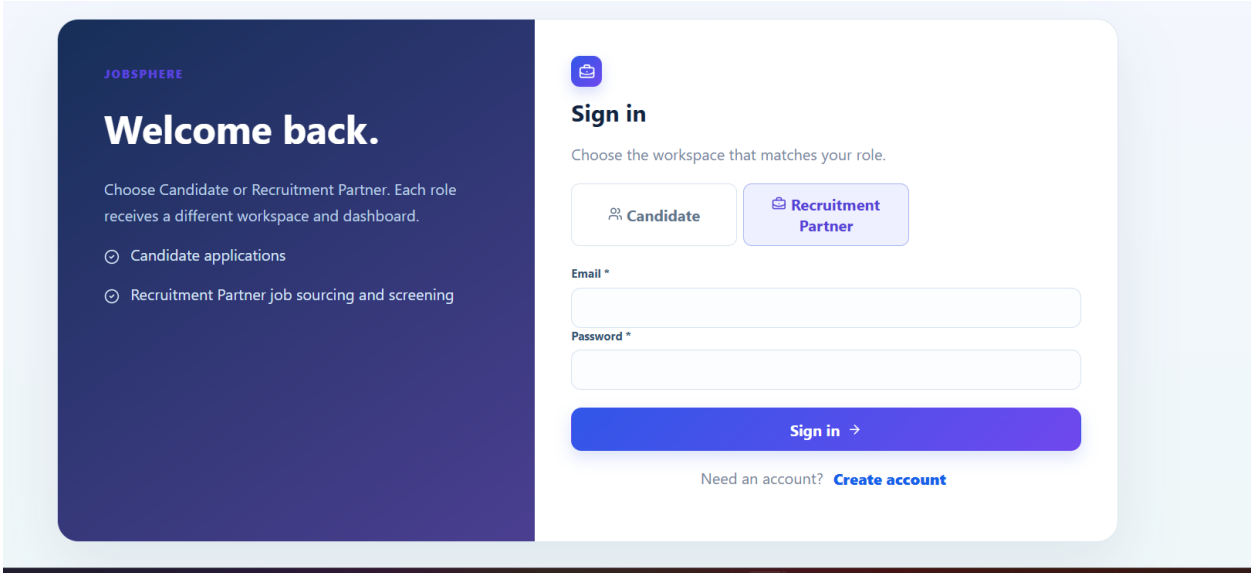


Figure 6.4 — Recruitment Partner Sign In.

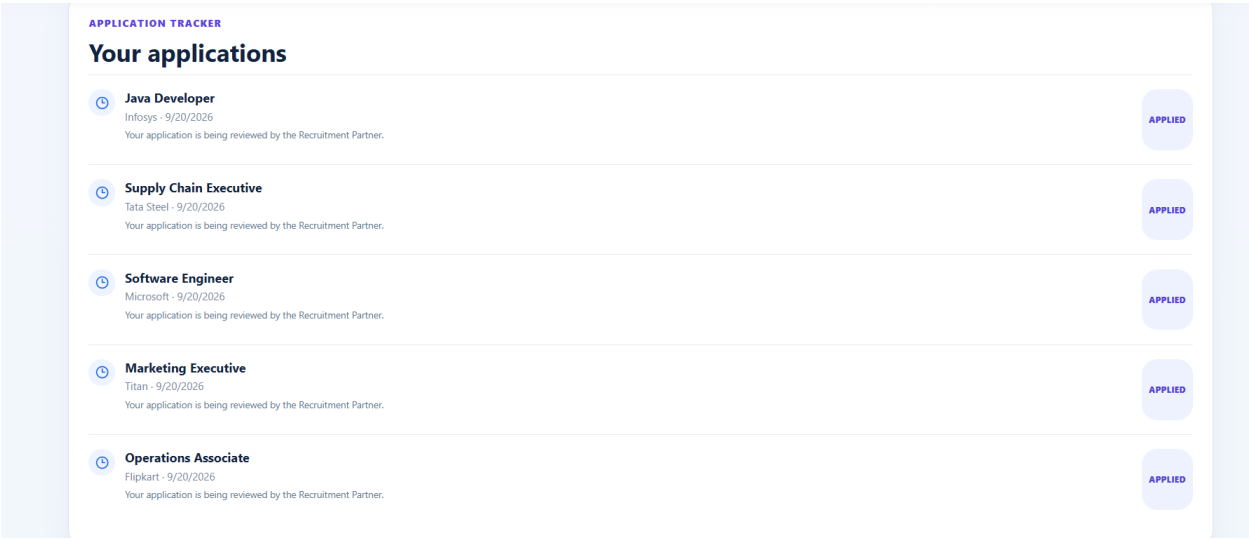


Figure 6.5 — Candidate Application Tracker.

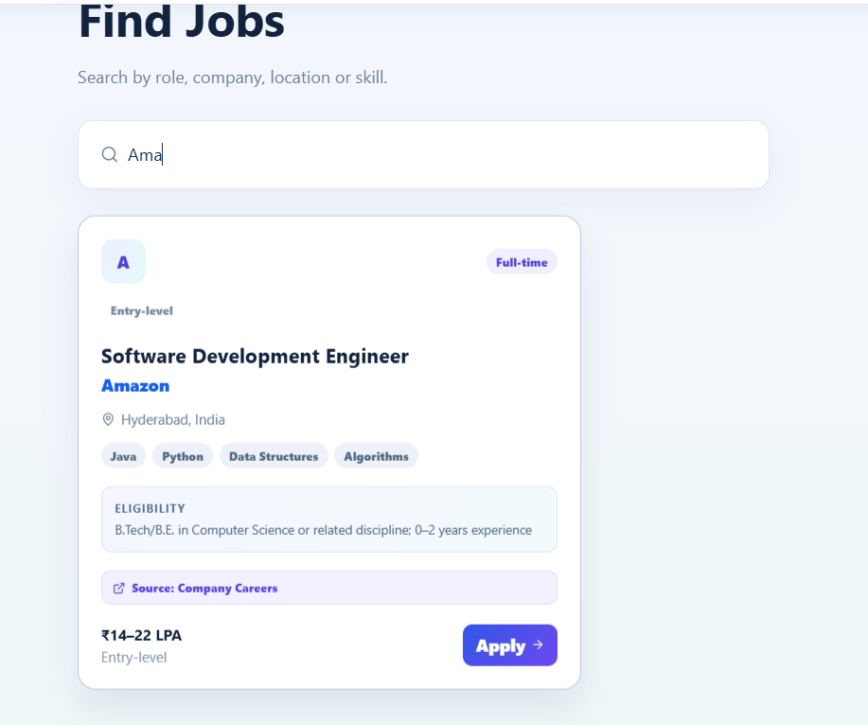


Figure 6.6 — Find Jobs.

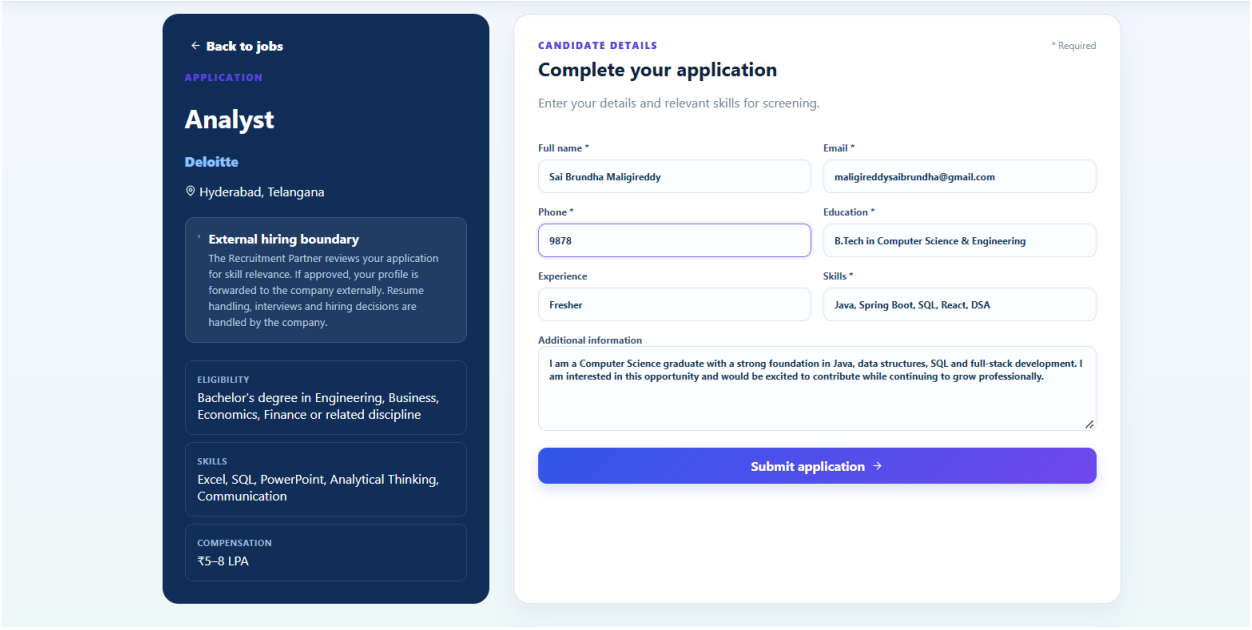


Figure 6.7 — Job Application.

7. Recruitment Partner Workflow

The Recruitment Partner workspace provides a separate navigation model for job sourcing and applicant review. The supplied recording demonstrates the recruiter landing/workspace, Build Your Skills, Manage Jobs and the Recruitment Partner Dashboard.

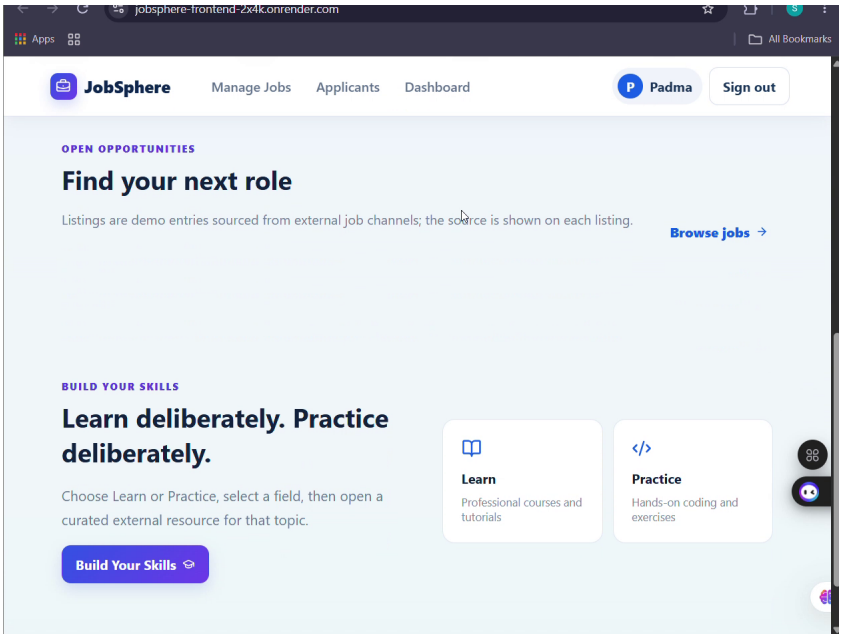


Figure 7.1 — Recruitment Partner workspace / landing interface.

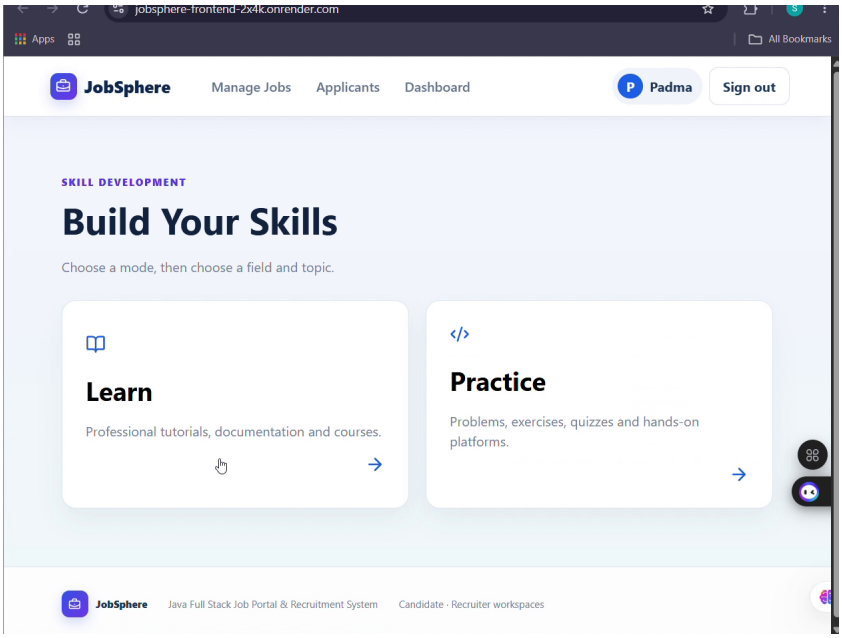


Figure 7.2 — Build Your Skills interface.

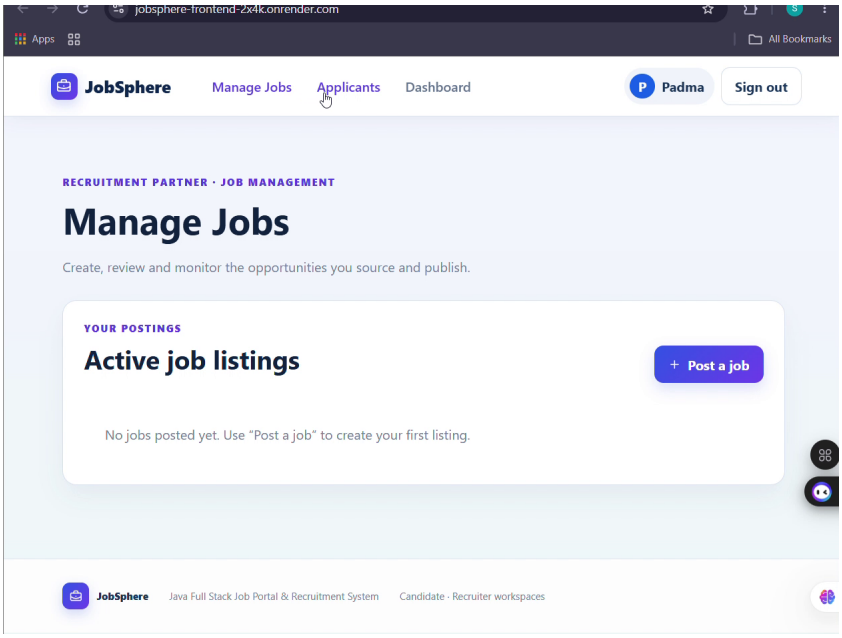


Figure 7.3 — Manage Jobs interface.

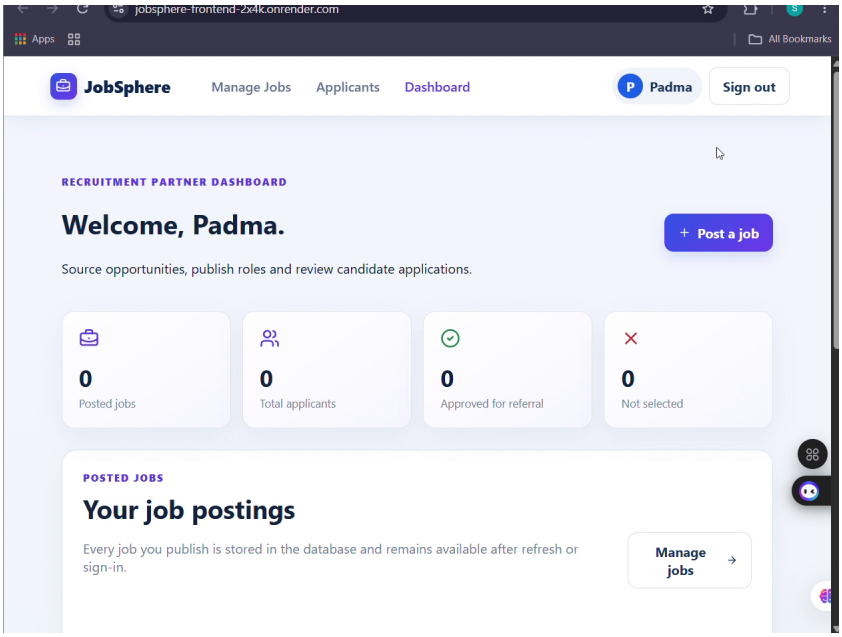


Figure 7.4 — Recruitment Partner Dashboard.

8. Company & Job Discovery

The Find Jobs screen supports search by role, company, location or skill. Job cards expose the position title, company, location, type, experience level, skills, eligibility criteria, source, compensation and application action.

The Companies directory is derived from active job data in the frontend workflow. Company profiles can present active openings and provide an external source link for the hiring process.

Figure 8.1 below shows the supplied search state with a Software Development Engineer opening at Amazon and its associated eligibility, skills, source and compensation information.

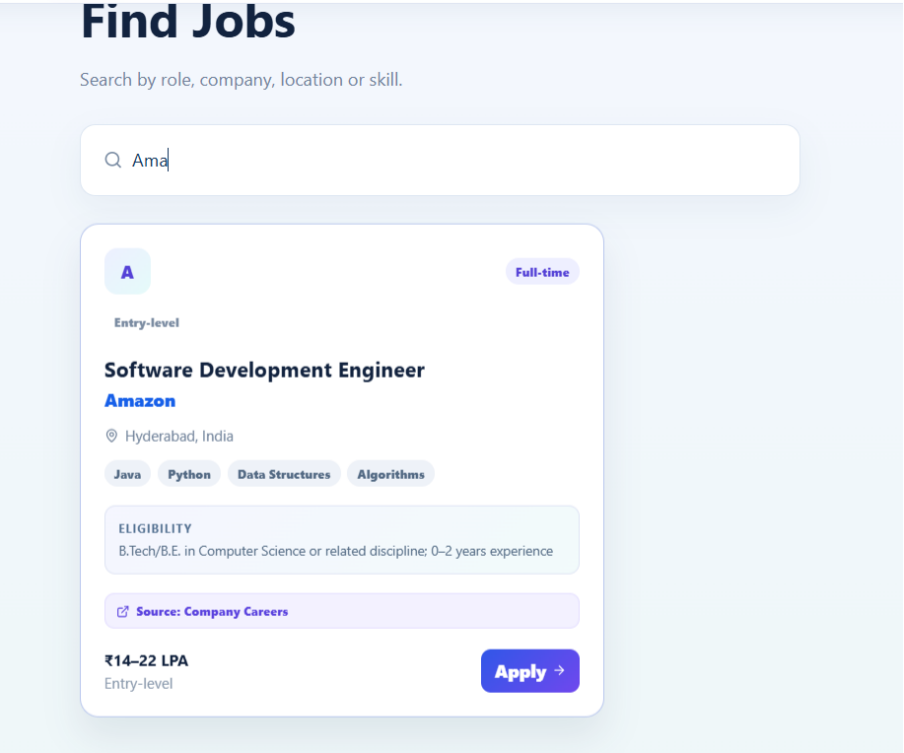


Figure 8.1 — Find Jobs search and job card.

9. Build Your Skills

Build Your Skills is a supporting learning area with two modes: Learn and Practice. Users choose a field and topic, then open a curated external resource. The implementation includes technology topics such as Java, Python, C++, Data Structures & Algorithms, Operating Systems, Computer Networks, DBMS & SQL, Compiler Design, Big Data, and Machine Learning & AI.

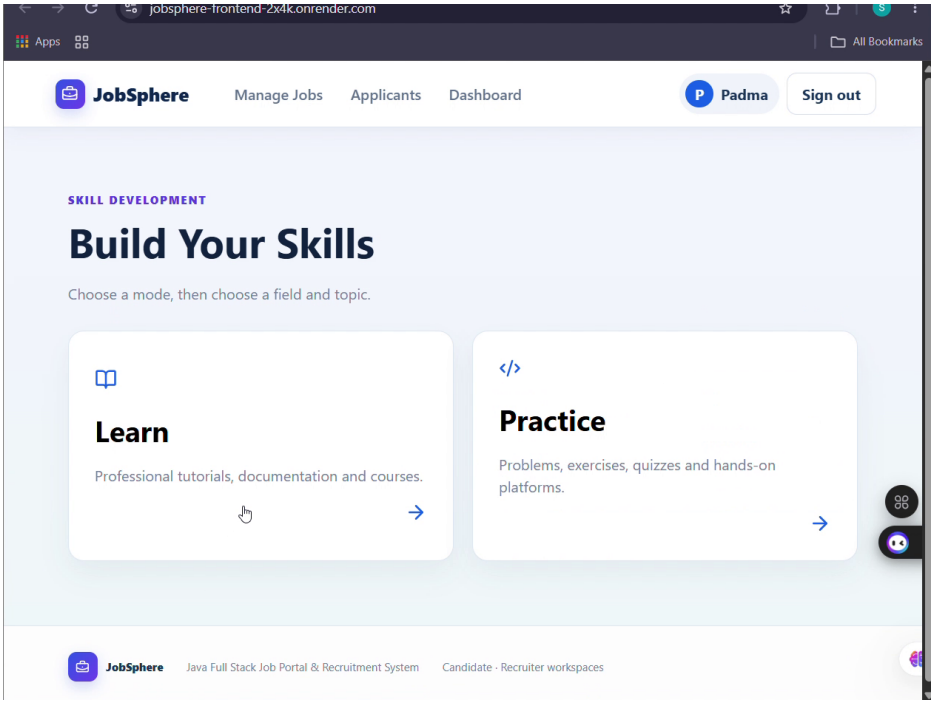


Figure 9.1 — Build Your Skills: Learn and Practice workspace.

The recording also shows the compact landing presentation of the learning area, where Learn and Practice are presented as two distinct paths.

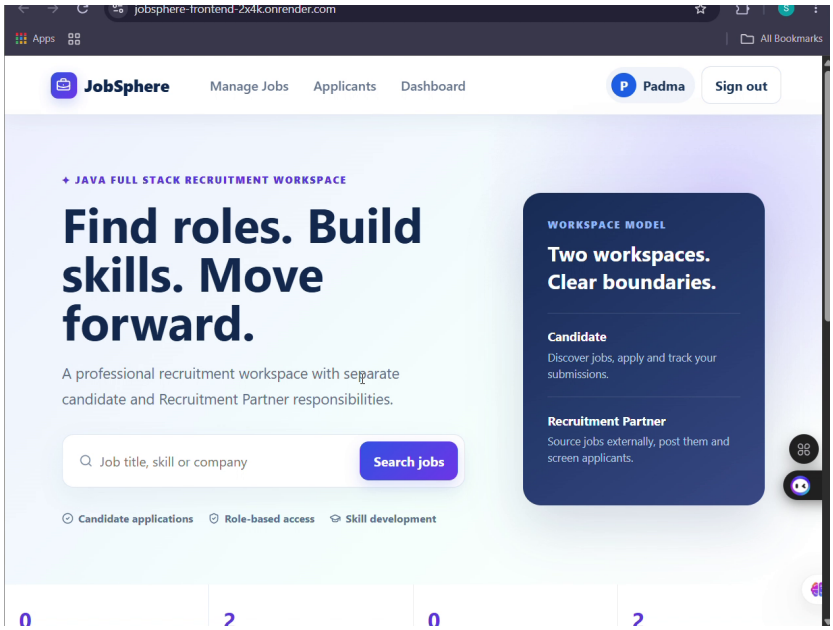


Figure 9.2 — JobSphere landing presentation with Build Your Skills section.

10. REST API & Data Model

Endpoint	Method	Purpose
/api/health	GET	Backend health check.
/api/auth/register	POST	Create a user account.
/api/auth/login	POST	Authenticate a user.
/api/jobs	GET / POST	List or create job postings.
/api/jobs/{id}	GET / DELETE	Retrieve or remove a job.
/api/jobs/recruiter/{id}	GET	Retrieve jobs belonging to a recruiter.
/api/applications	POST	Submit a candidate application.
/api/applications/candidate/{id}	GET	Retrieve candidate applications.
/api/applications/recruiter/{id}	GET	Retrieve recruiter applicants.
/api/applications/{id}/status	PATCH	Update application status.

Core entities include User, Job and Application. A recruiter can own job postings; a candidate can submit applications against jobs; applications retain applicant details, status, timestamp and screening information.

Entity	Important fields
User	id, name, email, password hash, role
Job	title, company, location, type, level, salary, source, eligibility, description, skills, active, createdAt, recruiter
Application	job, candidate, status, appliedAt, applicant name/email/phone, education, experience, skills, cover letter

11. Security, Validation & Demo Boundaries

Passwords are handled using BCrypt through Spring Security. Candidate application submission checks the candidate role and prevents duplicate applications to the same job. Recruiter-side status changes are ownership-aware in the service layer.

The application also includes startup schema-compatibility handling for fields such as source name, source URL, eligibility criteria, description and skills. A demo reset mechanism is present to support a clean demonstration dataset.

Important product boundary: the platform's application flow is designed around Recruitment Partner screening and external company hiring. Resume handling, interviews and final hiring decisions remain outside the demonstrated core workflow when the application is forwarded to the external company process.

For production hardening, controller-level authorization should be complemented by authenticated request context and stronger API-level authorization enforcement rather than relying on client-supplied identifiers alone.

12. Deployment Architecture

Component	Deployment role
React/Vite frontend	Static web application deployed through Render; API base URL is configured through VITE_API_URL.
Spring Boot backend	Docker-based service exposing REST APIs, with PostgreSQL connectivity.
PostgreSQL	Persistent relational database for users, jobs and applications.
Local development	Docker Compose configuration supports backend and frontend development workflow.

Live project reference: jobsphere-frontend-2x4k.onrender.com

Source repository: github.com/bru-reddy/Job_Portal_And_Recruitment_System

13. Interface Documentation

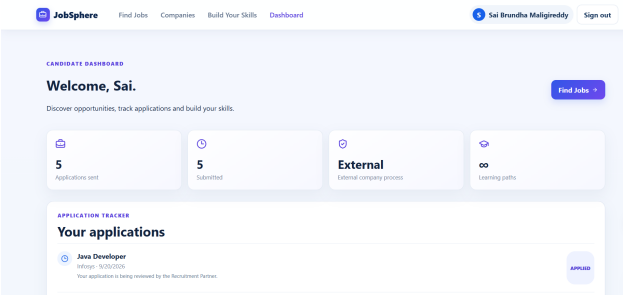


Figure 13.1 — Candidate Dashboard — overview cards and application tracker.

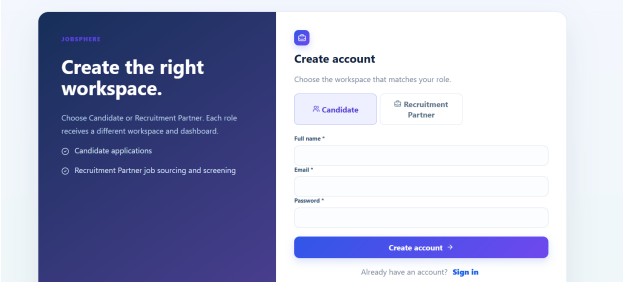


Figure 13.3 — Candidate Registration — account creation.

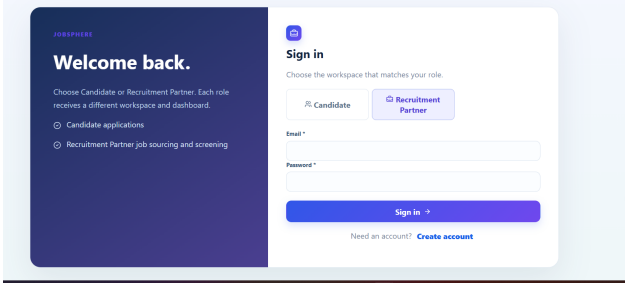


Figure 13.4 — Recruitment Partner Sign In — recruiter workspace selection.

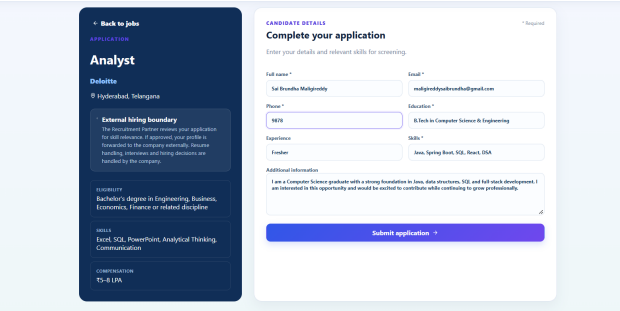


Figure 13.5 — Application Form — candidate details and screening fields.

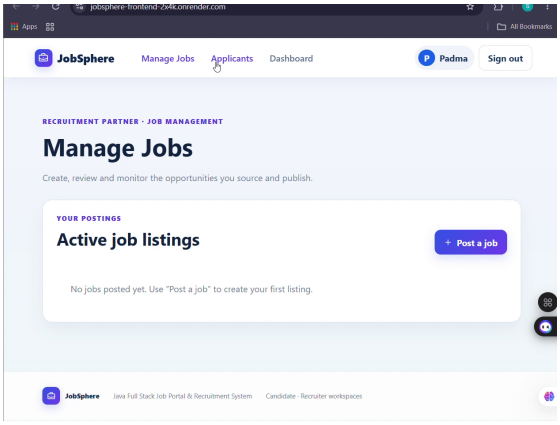


Figure 13.6 — Manage Jobs — recruitment partner job management.

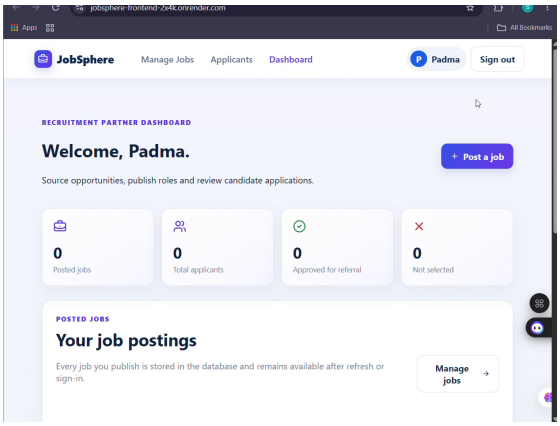


Figure 13.7 — Recruitment Partner Dashboard — recruiter overview.

14. Testing & Validation

Test area	Expected behavior	Evidence
Candidate authentication	Candidate role can be selected and credentials submitted.	Supplied sign-in and registration screens.
Recruiter authentication	Recruitment Partner workspace can be selected.	Supplied recruiter sign-in screen.
Job search	Search filters job listings by role/company/location/skill.	Find Jobs screenshot.
Application	Required candidate information is collected and submitted.	Application form screenshot.
Application tracking	Submitted applications appear with status and explanatory message.	Candidate dashboard/tracker screenshots.
Recruiter job management	Recruiter can access job-management workspace and posting action.	Recording frame of Manage Jobs.
Learning workspace	Learn and Practice modes are available.	Recording frame of Build Your Skills.
Responsive UI	Layouts adapt for mobile presentation.	Implementation includes responsive CSS; mobile evidence can be appended when supplied.

15. Future Enhancements

• Add resume upload and structured candidate profiles.

• Introduce richer recruiter filtering, sorting and candidate comparison.

• Add email or in-app notifications for application status changes.

• Implement interview scheduling and recruiter notes.

• Add stronger token/session-based API authorization and audit logging.

• Expand company-facing functionality beyond external source links.

• Add analytics for recruiter pipelines and candidate engagement.

• Introduce automated test coverage for API controllers, services and critical frontend workflows.

16. Conclusion & Project Links

JobSphere demonstrates an end-to-end recruitment workflow with distinct candidate and Recruitment Partner experiences. Its implementation combines a modern React interface, Spring Boot REST services, relational persistence, authentication, job discovery, application tracking, recruiter job management and skill-support resources.

Project	Reference
GitHub repository	https://github.com/bru-reddy/Job_Portal_And_Recruitment_System
Live frontend	https://jobsphere-frontend-2x4k.onrender.com

Documentation prepared from the supplied JobSphere interface screenshots, supplied screen recording, and the implemented project structure.